

1. 게임 기획 및 설계

이름	박수용	게임 제목	무인편의점 관리자
----	-----	-------	-----------

1. 게임 소개

본 프로젝트는 C언어를 활용하여 제작한 **선택형 시뮬레이션 게임**입니다.

게임 제목은 「무인편의점 관리자」로 정하였습니다.

이 게임은 플레이어가 무인편의점의 관리자가 되어 매장 안에서 발생하는 여러 상황에 대응하는 방식으로 진행됩니다. 최근 무인편의점, 무인카페, 무인매장과 같은 비대면 매장이 늘어나고 있다는 점에서 아이디어를 얻었습니다.

게임 속 플레이어는 CCTV를 통해 매장 상황을 확인하고, 손님의 행동이나 결제 문제, 도난 의심 상황 등에 알맞은 선택을 해야 합니다. 선택 결과에 따라 **보안 점수, 매출, 평판, 도난 위험도**가 변화하도록 설계하였습니다.

단순히 점수를 얻는 게임이 아니라, 보안을 너무 강하게 하면 고객 평판이 낮아질 수 있고, 반대로 너무 느슨하게 대응하면 도난 위험도가 높아지는 구조로 구성하였습니다. 이를 통해 플레이어가 상황에 따라 균형 잡힌 판단을 하도록 만들고자 하였습니다.

2. 게임 규칙

이 게임은 총 5턴 또는 7턴 동안 진행됩니다.

각 턴마다 무인편의점에서 발생할 수 있는 상황이 랜덤으로 출력됩니다.

플레이어는 출력된 상황을 읽고, 제시된 선택지 중 하나를 선택합니다. 선택지는 보통 1번, 2번, 3번으로 구성되며, 각 선택에 따라 결과가 달라집니다.

예를 들어 손님이 상품을 들고 결제하지 않은 채 출입구 쪽으로 이동하는 상황이 발생하면, 플레이어는 다음과 같은 선택을 할 수 있습니다.

- 바로 경찰에 신고한다.
- 경고 방송을 한다.
- 조금 더 지켜본다.

각 선택에 따라 보안 점수, 매출, 평판, 도난 위험도가 증가하거나 감소합니다.

게임의 기본 규칙은 다음과 같습니다.

1. 게임이 시작되면 플레이어 이름을 입력합니다.
2. 플레이어는 무인편의점 관리자가 됩니다.
3. 매 턴마다 랜덤 상황이 발생합니다.
4. 플레이어는 상황에 맞는 대응 방법을 선택합니다.
5. 선택 결과에 따라 상태값이 변화합니다.
6. 도난 위험도가 너무 높아지면 게임에 실패합니다.
7. 정해진 턴을 모두 버티면 최종 결과가 출력됩니다.
8. 최종 점수에 따라 S, A, B, C 등급을 출력합니다.

이 게임의 목표는 무인편의점에서 발생하는 문제를 적절히 해결하면서 **보안 점수와 매출, 평판을 높이고 도난 위험도를 낮게 유지하는 것**입니다.

3. 게임 흐름



4. 구현할 기능 목록

#	기능명	간단한 설명
1	메인 메뉴 기능	게임 시작, 게임 설명 보기, 종료를 선택할 수 있도록 구현하였습니다.
2	플레이어 정보 입력 기능	게임 시작 시 플레이어 이름을 입력받아 게임에 몰입할 수 있도록 구성하였습니다.
3	랜덤 상황 발생 기능	무인편의점에서 발생할 수 있는 여러 상황 중 하나가 무작위로 출력되도록 구현하였습니다.
4	선택지 입력 및 처리 기능	플레이어가 1, 2, 3번 중 하나를 선택하면 선택 결과에 따라 상태값이 변화하도록 구현하였습니다.
5	상태값 관리 기능	보안 점수, 매출, 평판, 도난 위험도 등의 값을 관리하도록 구현하였습니다.
6	게임 종료 판정 기능	도난 위험도가 일정 수치를 넘거나 정해진 턴이 끝나면 게임이 종료되도록 구현하였습니다.
7	최종 결과 출력 기능	최종 점수에 따라 S, A, B, C 등급을 출력하도록 구현하였습니다.
8	최고 점수 저장 기능	파일 입출력을 활용하여 최고 점수를 저장하고 다시 확인할 수 있도록 구현할 예정입니다.

5. 활용할 C 언어 문법

✓	C 언어 개념	활용 계획 메모
☑	배열 (Array)	여러 개의 상황 문장, 선택지, 이벤트 정보를 저장하는 데 활용하였습니다.
☑	함수 (Function)	메뉴 출력, 게임 진행, 상황 처리, 점수 계산, 결과 출력 기능을 각각 함수로 나누어 구현하였습니다.
☑	구조체 (struct)	플레이어의 이름, 보안 점수, 매출, 평판, 도난 위험도 등의 정보를 하나로 묶어 관리하였습니다.
☑	포인터 (Pointer)	함수에서 플레이어 상태값을 직접 변경할 때 구조체 포인터를 활용할 예정입니다.
☑	파일 입출력	최고 점수를 파일에 저장하고, 다음 실행 시 불러오는 기능에 활용할 예정입니다.
☑	반복문 / 조건문	게임 턴을 반복하고, 선택지와 종료 조건을 판단하는 데 활용하였습니다.
☑	기타: 랜덤 함수	rand() 함수를 사용하여 매 턴마다 다른 상황이 발생하도록 구현할 예정입니다.

6. 참조

1. C언어 수업 시간에 배운 배열, 함수, 구조체, 조건문, 반복문 개념을 참고하였습니다.
2. C언어의 rand() 함수를 활용한 랜덤 이벤트 발생 방식을 참고하였습니다.
3. 파일 입출력을 이용한 최고 점수 저장 방식을 참고하였습니다.
4. 실제 무인편의점에서 발생할 수 있는 도난, 결제 오류, 고객 응대 상황을 게임 아이디어로 참고하였습니다.
5. 콘솔 기반 선택형 게임 구조를 참고하여 게임 흐름을 설계하였습니다.

2. 구현 내용

6-1. 주요 기능 상세 설계

기능 1

기능명	메인 메뉴 기능
동작 설명	프로그램 실행 시 메인 메뉴 화면을 출력합니다. 플레이어가 게임 시작, 게임 설명 보기, 최고 점수 확인, 종료 중 하나를 선택할 수 있도록 구성하였습니다. 선택에 따라 해당 기능이 실행됩니다.
입력 / 출력	입력: 메뉴 번호(1~4) / 출력: 선택된 메뉴에 해당하는 화면
관련 함수명 (예정)	showMenu()

기능 2

기능명	플레이어 정보 입력 및 초기화 기능
동작 설명	게임 시작 시 플레이어의 이름을 입력받고, 보안 점수(50), 매출(100), 평판(50), 도난 위험도(30)를 초기값으로 설정합니다. 구조체 포인터를 활용하여 값을 직접 변경합니다.
입력 / 출력	입력: 플레이어 이름(문자열) / 출력: 초기화된 Player 구조체
관련 함수명 (예정)	initPlayer(Player *p)

기능 3

기능명	랜덤 상황 발생 기능
동작 설명	총 8개의 이벤트 중 5개를 중복 없이 랜덤으로 선택하여 출력합니다. Fisher-Yates 셔플 알고리즘을 사용하여 배열을 섞은 뒤, 앞에서 5개만 사용하는 방식으로 구현하였습니다.
입력 / 출력	입력: 이벤트 번호 배열 / 출력: 섞인 순서의 이벤트 배열
관련 함수명 (예정)	shuffleEvents(int arr[], int size)

기능 4

기능명	선택지 입력 및 처리 기능
동작 설명	각 상황에 대해 1, 2, 3번 선택지를 제시하고, 플레이어의 입력을 받아 처리합니다. 잘못된 입력(문자, 범위 초과)이 들어올 경우 입력 버퍼를 비우고 다시 입력받도록 구현하였습니다.
입력 / 출력	입력: 선택 번호(1~3) / 출력: 유효한 선택 번호 반환
관련 함수명 (예정)	getChoice()

기능 5

기능명	상태값 관리 및 이벤트 처리 기능
동작 설명	플레이어의 선택에 따라 보안 점수, 매출, 평판, 도난 위험도를 증가 또는 감소시킵니다. switch-case 문을 사용하여 8개 이벤트를 각각 처리하며, 구조체 포인터를 통해 값을 직접 변경합니다.
입력 / 출력	입력: Player 구조체 포인터, 이벤트 번호 / 출력: 변경된 상태값
관련 함수명 (예정)	handleEvent(Player *p, int eventNum)

기능 6

기능명	현재 상태 출력 기능
동작 설명	매 턴이 끝날 때마다 현재 보안 점수, 매출, 평판, 도난 위험도를 화면에 출력합니다. 플레이어가 자신의 상태를 확인하고 다음 선택에 참고할 수 있도록 합니다.
입력 / 출력	입력: Player 구조체(값 전달) / 출력: 상태값 콘솔 출력
관련 함수명 (예정)	showStatus(Player p)

기능 7

기능명	게임 종료 판정 및 결과 출력 기능
동작 설명	도난 위험도가 100 이상이거나 평판이 0 이하가 되면 게임이 실패로 종료됩니다. 5턴을 모두 버틴 경우 최종 점수를 계산하고, S/A/B/C 등급을 출력합니다.
입력 / 출력	입력: Player 구조체(값 전달) / 출력: 등급 및 최종 점수 콘솔 출력
관련 함수명 (예정)	showResult(Player p)

기능 8

기능명	최고 점수 저장 및 불러오기 기능
동작 설명	게임 종료 시 최종 점수가 기존 최고 점수보다 높으면 파일(highscore.txt)에 저장합니다. 파일이 없는 경우에도 오류 없이 동작하도록 fopen 반환값을 확인하는 방식으로 처리하였습니다.
입력 / 출력	입력: Player 구조체 / 출력: highsore.txt 파일 생성 또는 갱신
관련 함수명 (예정)	saveHighScore(Player p), loadHighScore()

6-2. 함수 목록 설계

함수명	반환형	매개변수	역할 설명
showMenu	void	없음	메인 메뉴 화면을 출력합니다.
showGameInfo	void	없음	게임 설명(규칙, 목표)을 출력합니다.
initPlayer	void	Player *p	플레이어 이름 입력 및 상태값 초기화를 수행합니다.
showStatus	void	Player *p	현재 보안, 매출, 평판, 위험도를 출력합니다.
playGame	void	Player *p	게임 전체 흐름을 제어합니다. (셔플 → 턴 반복 → 종료 판정)
handleEvent	void	Player *p, int eventNum	이벤트 번호에 따른 상황 출력 및 선택 결과를 처리합니다.
getChoice	int	없음	1~3 범위의 유효한 입력을 받아 반환합니다. 잘못된 입력 시 재입력을 요청합니다.
shuffleEvents	void	nt arr[], int size	Fisher-Yates 셔플로 이벤트 배열을 랜덤하게 섞습니다.
showResult	void	Player p	최종 점수 계산 및 S/A/B/C 등급을 출력합니다.
saveHighScore	void	Player p	최고 점수를 파일(highscore.txt)에 저장합니다.
loadHighScore	void	없음	파일에서 최고 점수를 읽어 출력합니다.

6-3. 자료구조 설계

구조체 정의

구조체명	멤버 변수	자료형	역할 설명
Player	name	char[30]	플레이어의 이름을 저장합니다.
	security	int	보안 점수를 저장합니다. (초기값: 50)
	sales	int	매출을 저장합니다. (초기값: 100)
	reputation	int	평판을 저장합니다. (초기값: 50)
	risk	int	도난 위험도를 저장합니다. (초기값: 30)
	score	int	최종 점수를 저장합니다.

사용할 전역 변수 / 배열

자료형	변수명	역할 설명
int	order[TOTAL_EVENTS]	이벤트 순서를 저장하는 배열입니다. {0,1,2,...,7}로 초기화한 뒤 셔플합니다.
#define	MAX_TURNS (5)	한 게임당 진행할 최대 턴 수를 정의합니다.
#define	TOTAL_EVENTS (8)	전체 이벤트 개수를 정의합니다.
#define	RISK_LIMIT (100)	도난 위험도 실패 기준값을 정의합니다.
#define	REP_LIMIT (0)	평판 실패 기준값을 정의합니다.
#define	SAVE_FILE	최고 점수 저장 파일명("highscore.txt")을 정의합니다.

6-4. C 언어 문법 활용 계획 (구체적)

C 언어 개념	구체적인 활용 방법 (어느 기능에, 어떻게 적용하는지)
배열 (Array)	이벤트 순서를 저장하는 order[TOTAL_EVENTS] 배열을 선언하여 {0,1,2,...,7}로 초기화하고, 셔플 후 앞에서 5개를 꺼내 사용하였습니다. 배열을 사용함으로써 8개의 이벤트를 인덱스로 관리할 수 있었습니다.
함수 (Function)	프로그램 기능을 11개의 함수로 분리하였습니다. showMenu(), playGame(), handleEvent() 등 각 기능별로 함수를 나누어 코드의 가독성과 유지보수성을 높였습니다.
구조체 (struct)	플레이어의 이름, 보안 점수, 매출, 평판, 도난 위험도, 최종 점수를 하나의 Player 구조체로 묶어 관리하였습니다. typedef를 사용하여 구조체 이름을 간결하게 사용할 수 있도록 하였습니다.
포인터 (Pointer)	initPlayer(Player *p), playGame(Player *p), handleEvent(Player *p, int eventNum) 함수에서 구조체 포인터를 사용하여 함수 내부에서 원본 데이터를 직접 변경할 수 있도록 하였습니다. 화살표 연산자(->)를 활용하였습니다.
파일 입출력	fopen(), fprintf(), fscanf(), fclose()를 사용하여 최고 점수를 highscore.txt 파일에 저장하고, 다음 실행 시 불러오도록 구현하였습니다. 파일이 존재하지 않을 때는 "기록 없음"을 출력하도록 예외 처리하였습니다.
반복문 (for/while)	for문으로 게임 턴을 반복(5회)하고, while문으로 잘못된 입력 시 재입력을 받도록 구현하였습니다. Fisher-Yates 셔플에서도 for문을 활용하였습니다.
조건문 (if/switch)	if문으로 게임 종료 조건(위험도 100 이상, 평판 0 이하)을 판단하고, switch-case문으로 8개 이벤트를 각각 분기 처리하였습니다. 메인 메뉴 선택에서도 switch를 사용하였습니다.
문자열 (string.h)	플레이어 이름 저장에 char 배열과 scanf를 사용하였습니다. 출력 시 %s 서식 지정자를 활용하여 이름을 포함한 메시지를 출력하였습니다.
매크로 (#define)	MAX_TURNS, TOTAL_EVENTS, RISK_LIMIT, REP_LIMIT, SAVE_FILE 등 상수를 매크로로 정의하여 코드 수정이 용이하도록 하였습니다.
랜덤 함수 (rand)	srand(time(NULL))로 시드를 설정하고, rand() % size를 사용하여 이벤트 배열을 셔플하였습니다. 매 실행마다 다른 이벤트 순서가 나오도록 구현하였습니다.

체크리스트

☐	게임의 목적과 규칙이 명확하게 정의되었다.
☐	전체 흐름(플로우차트)이 논리적으로 구성되었다.
☐	구현할 기능이 3가지 이상 구체적으로 작성되었다.
☐	각 기능에 대응하는 함수가 설계되어 있다.
☐	사용할 자료구조(변수, 배열 등)가 정리되었다.
☐	수업에서 배운 C 언어 문법이 다양하게 활용될 예정이다.